

Buổi 2 – Luyện tập nâng cao với Git

🎯 Mục tiêu buổi học

- Tiếp tục thực hành trên repo đã tạo ở Buổi 1 (git-practice).
- Làm quen với các lệnh Git nâng cao: stash, reset, revert, tag, rebase.
- Hiểu sự khác nhau giữa các phương pháp quản lý lịch sử commit.
- Thực hành các tình huống sát với làm việc nhóm thực tế.

Repo sử dụng

- Mỗi sinh viên sẽ làm việc trên repo cá nhân đã tạo ở Buổi 1: <https://github.com/<your-username>/git-practice>
- Toàn bộ hoạt động trong Buổi 2 được thực hiện trên repo này.
- Nếu muốn giữ repo gốc sạch sẽ, sinh viên có thể tạo thêm repo mới tên git-practice-advanced.

Hoạt động 1: Ôn tập cơ bản với nhiều tình huống

👉 Mục tiêu: Hiểu sự khác biệt giữa Working Directory, Staging Area và Repository.

Bối cảnh: Giả sử bạn đang làm việc trên nhánh main của repo git-practice. Để tránh ảnh hưởng code chính, bạn cần thực hành trên nhánh riêng.

Yêu cầu chi tiết:

1. Tạo nhánh mới từ main để thực hành:
`git checkout -b practice-branch`
2. Trong nhánh này, tạo 3 file:
 - practice1.txt → viết: "File cho staging + commit"
 - practice2.txt → viết: "File cho staging, chưa commit"
 - practice3.txt → viết: "File ở working directory"
3. Thao tác:
 - `git add practice1.txt` → commit với message "Add practice1".
 - `git add practice2.txt` → chưa commit.
 - Để nguyên practice3.txt trong working directory.
4. Quan sát:
 - `git status`
 - `git diff`
 - `git diff --staged`

5. Thực hành khôi phục:
 - git restore practice3.txt
 - git restore --staged practice2.txt
6. Quản lý file:
 - git mv practice1.txt exercise1.txt
 - git rm exercise1.txt
7. Đẩy nhánh practice-branch lên GitHub.
8. Tạo Pull Request vào main trên GitHub.

Hoạt động 2: Sử dụng git stash

👉 Mục tiêu: Lưu thay đổi tạm thời khi chưa muốn commit.

Bối cảnh: Bạn đang chỉnh sửa README.md thì được yêu cầu pull code mới từ nhóm.

Yêu cầu chi tiết:

1. Chỉnh sửa file README.md nhưng không commit.
2. git stash save "update readme"
3. git stash list
4. Tạo file note.md, commit và push.
5. git stash pop
6. Commit và push thay đổi.
7. Stash 2 lần liên tiếp.
8. git stash apply stash@{0}
9. git stash drop stash@{0}
10. git stash clear

Hoạt động 3: Revert & Reset

👉 Mục tiêu: Hiểu cách quay lại trạng thái trước đó bằng nhiều cách.

1. Tạo file history.txt và tạo 3 commit (dòng 1, dòng 2, dòng 3).
2. git log --oneline
3. git revert commit thứ 3.
4. Tạo thêm 2 commit mới.
5. Thực hiện reset:
 - git reset --soft <commit-1>
 - git reset --mixed <commit-1>
 - git reset --hard <commit-1>
6. git reflog
7. Khôi phục commit đã mất bằng git checkout hoặc git reset.

Hoạt động 4: Tag và Release

👉 Mục tiêu: Biết cách đánh dấu mốc quan trọng của dự án.

1. Tạo file version.txt ghi 'Version 1.0'.
2. `git commit -m "Add version 1.0"`
3. `git tag v1.0`
4. `git push origin v1.0`
5. Chỉnh sửa version.txt thành 'Version 2.0'.
6. `git commit -m "Update version 2.0"`
7. `git tag -a v2.0 -m "Release version 2.0"`
8. `git push origin v2.0`
9. `git tag -l`
10. `git tag beta-1.0`
11. `git tag -d beta-1.0`
12. `git push origin --delete v1.0`
13. `git checkout -b branch-from-v2 v2.0`

Hoạt động 5: Rebase thay vì merge

👉 Mục tiêu: Làm lịch sử commit gọn gàng hơn.

1. Tạo nhánh feature-branch và commit file feature.md.
2. Quay lại main và commit file main.md.
3. Merge feature-branch:
`git merge feature-branch`
`git log --graph --oneline`
4. Reset lại main về trước khi merge.
5. `git rebase feature-branch`
`git log --graph --oneline`
6. Tạo commit mới ở cả main và feature-branch.
7. Thực hiện rebase lại → conflict.
8. Giải quyết conflict, `git rebase --continue`.
9. Viết 3-5 câu so sánh merge và rebase.

Yêu cầu nộp sau Buổi 2

- File Word ghi lại các lệnh đã chạy, screenshot kết quả, và giải thích ngắn.
- Link GitHub repo sau buổi 2 (có nhiều commit, nhánh, tag, rebase).